

Chapter-6

Decision Making and Looping

Why?

➤ To print “hello” 10 times
write 10 times **printf(“hello”);**

- Read and Display 5 numbers.

```
int a,b,c,d,e;  
scanf(“%d”,&a);  
scanf(“%d”,&b);  
scanf(“%d”,&c);  
scanf(“%d”,&d);  
scanf(“%d”,&e);
```

OR

```
scanf(“%d%d%d%d%d”,&a,&b,&c,&d,&e);
```

Cont..

```
printf(“%d”,a);  
printf(“%d”,b);  
printf(“%d”,c);  
printf(“%d”,d);  
printf(“%d”,e);
```

OR

```
printf(“%d%d%d%d%d”,a,b,c,d,e);
```

Using goto print hello 10 times

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    int count,a=1;
```

```
    printf("enter number of times you want to print hello:");
```

```
    scanf("%d",&count); //10
```

```
    abc:
```

```
        printf("hello\n");
```

```
        a=a+1;
```

```
        if(a<=count)
```

```
        {
```

```
            goto abc;
```

```
        }
```

```
}
```

Why Looping?

- We can execute a segment of program repeatedly without using goto statement.
- There are more convenient method in C than the goto statement and that is looping in C.
- These looping enable us to develop concise programs containing repetitive processes without the use of goto statements.
- In looping, a sequence of statements are executed until some conditions for termination of the loop are satisfied.

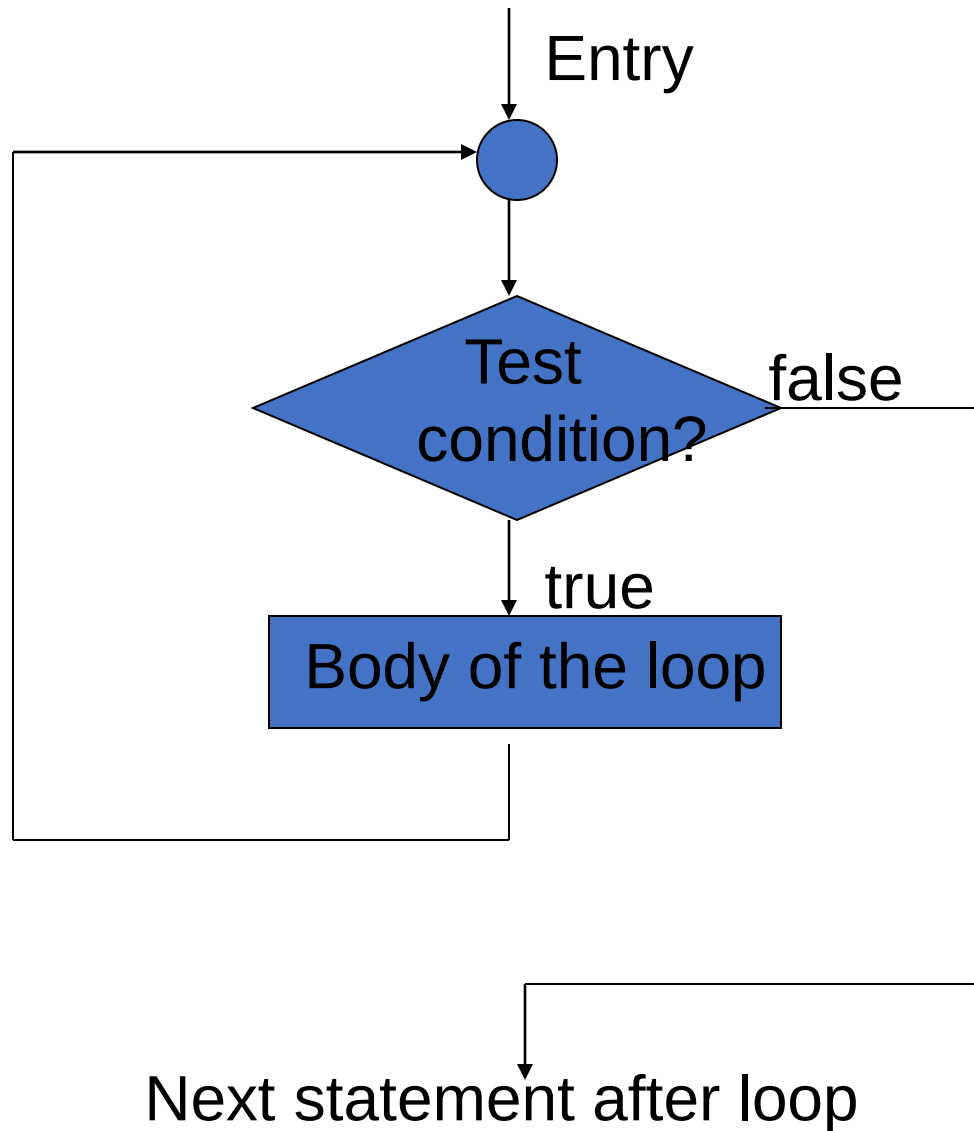
Cont..

- Loop consists of two segments
 - 1) Body of loop
 - 2) control statement
- Control statement tests certain conditions and if true then directs the control to the body of loop.

Loop Control Structure

- Depending on the position of the control statement in the loop, a control structure classified as
 - 1) entry-controlled loop
 - 2) exit-controlled loop.
- In **entry-controlled loop** Control conditions are tested before the start of the loop.
- If condition true then body of the loop is executed. If condition false then body of the loop will not be executed.
- while & for loop.
- Entry-controlled loops are also known as pre-test loops.

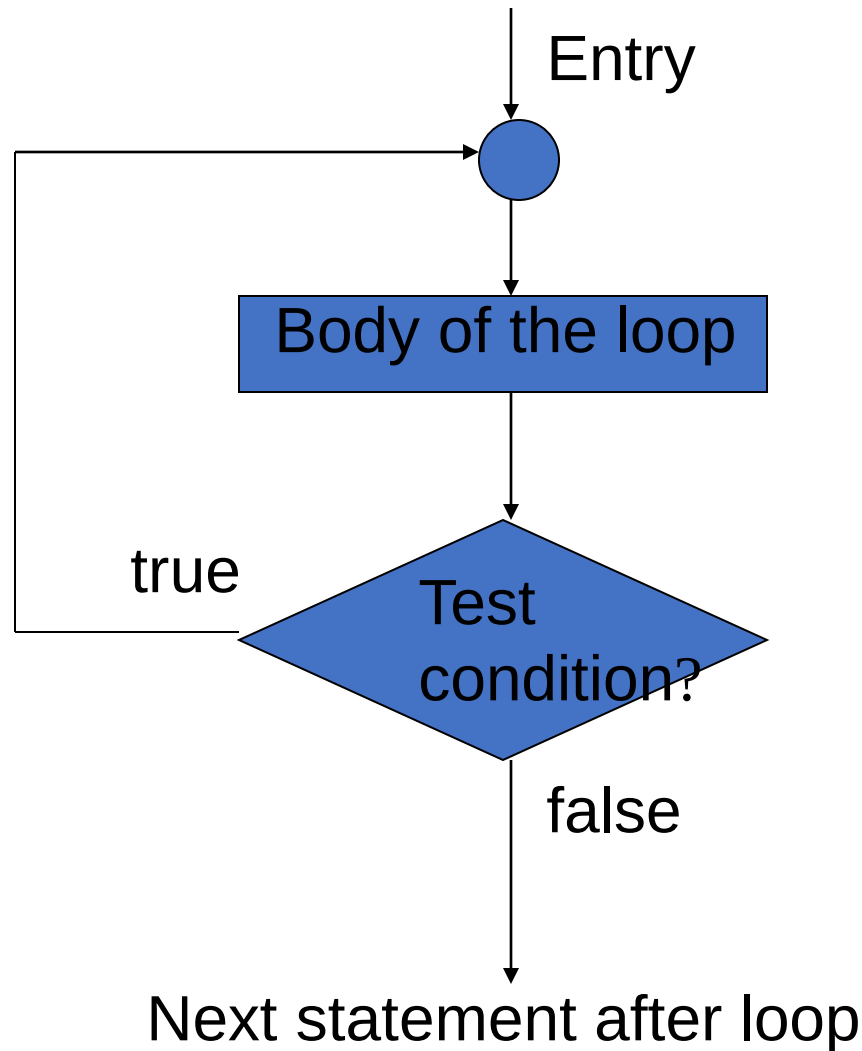
Flowchart-Entry Control Loop



Cont..

- In case of an **exit-controlled loop**, the test is performed at the end of the body of the loop and therefore the body is executed unconditionally for the first time.
- do-while loop
- Exit-controlled loops are also known as post-test loops.

Flowchart-Exit Control Loop



Test Condition

- The test conditions should be carefully stated because,
- If due to some reason it does not terminate the loop, the control sets up an infinite loop and the body is executed over and over again.

Looping Process

A looping process would include the following four steps:-

- Setting and initialization of counter
- Test for a specified condition for execution of the loop
- Execution of the statements in the loop
- Incrementing the counter

Counter-controlled loops vs Sentinel-controlled loops

- When we know in advance exactly how many times the loop will be executed, we use a counter-controlled loop.
- We use a control variable known as counter. The counter must be initialized, tested and updated properly for the desired loop operations.
- The number of times we want to execute the loop may be a constant or a variable that is assigned a value.
- A counter-controlled loop is sometimes called definite repetition loop.

Cont...

- In a sentinel-controlled loop, a special value called a sentinel value is used to change the loop control expression from true to false.
- For example, when reading data we may indicate the "end of data" by a special value, like -1 and 999. The control variable is called sentinel variable.
- A sentinel-controlled loop is often called indefinite repetition loop because the number of repetitions is not known before the loop begins executing

Types of Loop

- while loop
- do while loop
- for loop

while Statement

- while is an entry-controlled loop statement

Format of the while statement

```
while(test-condition)
{
    body of the loop;
}
```

- Body of the loop may have one or more statements.
- Braces are needed only if the body contains two or more statements.

Example: Print “hello” 10 times.

```
#include<stdio.h>

void main()
{
    int count,a=1;
    printf("enter number of times you want to print hello:");
    scanf("%d",&count); //10
    while(a<=count)
    {
        printf("hello\n");
        a=a+1;
    }
}
```

Example: Print 1 to10

```
#include<stdio.h>

void main()
{
    int n=1;
    while(n<=10)
    {
        printf("%d\n",n);
        n=n+1;
    }
}
```

Sum of first n number using while

```
#include<stdio.h>
void main()
{
    int a=1,sum=0,n;
    scanf("%d",&n);
    while(a<=n)    //counter controlled loop
    {
        sum=sum+a;
        a=a+1;
    }
    printf("%d\n",sum);
}
```

while loop-as a sentinel controlled loop

```
#include<stdio.h>
void main()
{
    char ch;
    ch=' ';
    while(ch!='y')  //sentinel controlled
    {
        ch=getchar();
        printf("%c",ch);
    }
}
```

do-while Loop

- do statement is an exit controlled loop statement.

Format of do statement

```
do  
{  
    body of the loop;  
} while (test-condition);
```

Example: Print 1 to10

```
#include<stdio.h>
void main()
{
    int n=1;
    do
    {
        printf(“%d\n”, n);
        n = n+1;
    } while (n<=10); // counter controlled
}
```

do-while loop-as a sentinel controlled loop

```
#include<stdio.h>
void main()
{
    int sum=0,n;
    scanf("%d",&n);
    do
    {
        sum=sum+n;
        scanf("%d",&n);
    }while(n>0); // sentinel controlled
    printf("%d",sum);
}
```

For Loop

- for loop is an entry-controlled loop

General form of for loop

```
for(initialization ; test-condition ; increment/decrement)
{
    body of the loop;
}
```

Example: Print “hello” 10 times.

```
#include<stdio.h>

void main()
{
    int count,i;
    printf("enter number of times you want to print hello:");
    scanf("%d",&count); //10
    for(i=0;i<count;i++)
    {
        printf("hello\n");
    }
}
```

Example: Print 1 to10

```
#include<stdio.h>
void main()
{
    int n,i;
    scanf("%d",&n); //10
    for(i=1;i<=n;i=i+1) // counter controlled
    {
        printf("%d\n",i);
    }
}
```

for loop-as a sentinel controlled loop

➤sum of n random number till sum>100

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
int n,i,sum=0;
```

```
for(i=1;sum<=100;i=i+1) //sentinel controlled
```

```
{
```

```
scanf("%d",&n);
```

```
sum=sum+n;
```

```
}
```

```
printf("%d",sum);
```

```
}
```

Decrement Counter

- ```
for (x =9; x >= 0; x = x -1)
 printf ("%d\n", x);
```
- for loop is executed 10 times and prints the digits 9 to 0.
- Braces are optional when the body of the loop contains only one statement.
- What will be the output of following code:

```
for (x=9; x< 9;x=x-1)
{
 printf ("%d", x);
}
```

# Additional Features Of For Loop

- More than one variable can be initialized in for loop

**ex.**

```
p=1;
for (n=0;n<17;n++)
```

Can be rewritten as:

```
for (p=1,n=0; n<17; n++)
```

- Increment section also have more than one part.

```
for (n=1, m=50; n<=m; n=n+1, m=m-1)
{
 p=m/n;
 printf(“%d %d %d\n”,n,m,p);
}
```

# Cont...

- Test-condition may have any compound relation.

```
for(i=1; i<20 && sum<100; ++i)
{
 sum = sum + i;
 printf(“%d %d\n”,i,sum);
}
```

- It is also permissible to use expressions in the assignment statements of initialization and increment sections.

```
for(x = (m+n)/2; x>0; x=x/2)
```

# Cont..

- We can also omit one or more sections if necessary.

```
m = 5;
for (; m !=100 ;)
{
 printf(“%d\n”, m);
 m = m+5;
}
```

- If the test-condition is not present, the for statement sets up an infinite loop.

```
for(; ;)
{

}
```

# Cont..

- We can set up time delay using the null statement as follows,

```
for(j = 1000; j>0; j = j-1)
;
```

- This loop is executed 1000 times without producing any output, It simply causes a time delay.

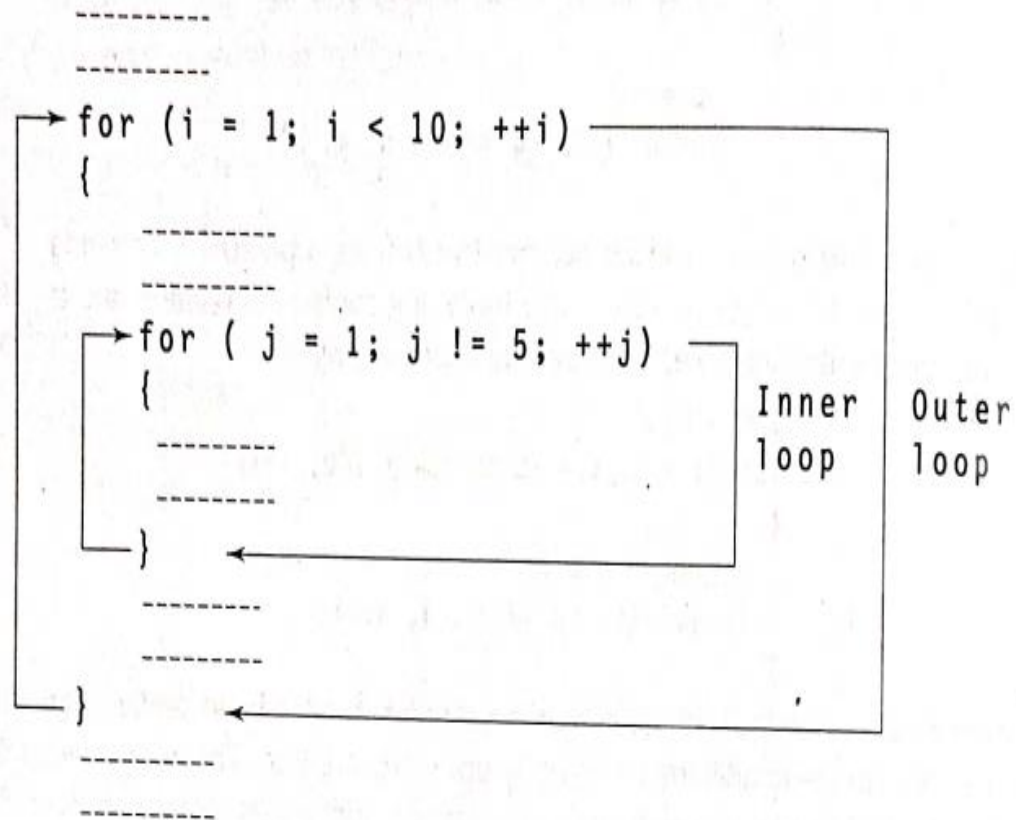
- This can also be written as follows:

```
for(j = 1000; j>0; j = j-1);
```

- Put semicolon if and only if you want to generate time delay.

# Nesting of for loops

- One for statement within another for statement, is allowed in C.
- ANSI C allows upto 15 level of nesting.



**Write a program to print following matrix:**

|   |    |    |    |
|---|----|----|----|
| 1 | 2  | 3  | 4  |
| 5 | 6  | 7  | 8  |
| 9 | 10 | 11 | 12 |

# Without nesting of for loops

```
for (i=1 ; i<=4 ; i++)
{
 printf ("%d\t",i);
}
printf("\n");
```

```
for (i=5 ; i<=8 ; i++)
{
 printf ("%d\t",i);
}
printf("\n");
```

```
for (i=9 ; i<=12 ; i++)
{
 printf ("%d\t",i);
}
```

# Using nesting of for loops

```
int n=1,i,j;
for (i=1 ; i<=3 ; i++)
{
 for (j=1 ; j<=4 ; j++)
 {
 printf(“%d\t”,n);
 n++;
 }
 printf (“\n”);
}
```

# Selecting a loop

- Analyse the problem and see whether it required a pre-test or post-test loop.
- If it requires a post-test loop, then we can use only one loop, do while.
- If it requires a pre-test loop, then we have two choices: for and while.
- Decide whether the loop termination requires counter-based control or sentinel-based control.
- Use **for loop** if the counter-based control is necessary.

# Cont...

- Use **while loop** if the sentinel-based control is required.
- Note that both the counter-controlled and sentinel-controlled loops can be implemented by all the **three control structures**.

# C Program to print factorial of a number

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
 int num, count, fact = 1;
```

```
 printf("Enter a number to find its Factorial:");
```

```
 scanf("%d", &num); //4
```

```
 for(count=2; count <= num; count++)//=num;count>=2;count--
```

```
 {
```

```
 fact = fact * count;
```

```
 }
```

```
 printf("Factorial of %d is %d\n", num, fact);
```

```
}
```

# factorial

$$5!=5*4*3*2*1$$

$$4!=4*3*2*1$$

$$=1*2*3*4$$

$$\text{fact}=\text{fact}*\text{count}$$

$$=1*2=2$$

$$\text{fact}=\text{fact}*\text{count}$$

$$=2*3=6$$

$$\text{fact}=\text{fact}*\text{count}$$

$$=6*4=24$$

# C Program to print Fibonacci Series

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
 int a=0,b=1,c,n,i;
```

```
 printf("Enter the number of terms to be printed: ");
```

```
 scanf("%d",&n); //7
```

```
 printf("%d\t",a);
```

```
 printf("%d\t",b);
```

```
 for(i=3 ; i<=n ; i++) //i=1;i<=n-2;i++
```

```
 {
```

```
 c = a + b;
```

```
 printf("%d\t",c);
```

```
 a=b;
```

```
 b=c; }
```

```
}
```

# Fibonacci Series

0,1,1,2,3,5,8,13,21.....

$n=5$

0,1,1,2,3

$n=7$

0,1,1,2,3,5,8

# C Program to print sum of digits of a number

```
#include<stdio.h>

void main()
{
 int n,d,sum=0,y;
 printf("Enter a number: ");
 scanf("%d",&n);
 y=n;
 while(n>0)
 {
 d=n%10;
 sum=sum+d;
 n=n/10; }
 printf("The sum of digits of %d is %d.",y,sum);
}
```

# sum of digits of a number

d=n%10;

sum=sum+d;

n=n/10;

//123=6 5625=18 28=10 13342=13

**123**

$123 \% 10 = 3$

$\text{Sum} = 0 + 3 = 3$

$n = 123 / 10 = 12$

$12 \% 10 = 2$

$\text{Sum} = 3 + 2 = 5$

$n = 12 / 10 = 1$

$1 \% 10 = 1$

$\text{sum} = 5 + 1 = 6$

$n = 1 / 10 = 0$

# C Program to print reverse of a number

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
 int n,d,rev=0,y;
```

```
 printf("Enter a number: ");
```

```
 scanf("%d",&n);
```

```
 y=n;
```

```
 while(n>0)
```

```
 {
```

```
 d=n%10;
```

```
 rev=rev*10+d;
```

```
 n = n / 10;
```

```
 }
```

```
 printf("The sum of digits of %d is %d.",y,rev); // Using the copy
```

```
}
```

# reverse of a number

**d=n%10;**

**rev=rev\*10+d;**

**n = n / 10;**

**123**

$123 \% 10 = 3$

$rev = 0 * 10 + 3 = 3$

$n = 123 / 10 = 12$

$12 \% 10 = 2$

$rev = 3 * 10 + 2 = 32$

$n = 12 / 10 = 1$

$1 \% 10 = 1$

$rev = 32 * 10 + 1 = 321$

$n = 1 / 10 = 0$

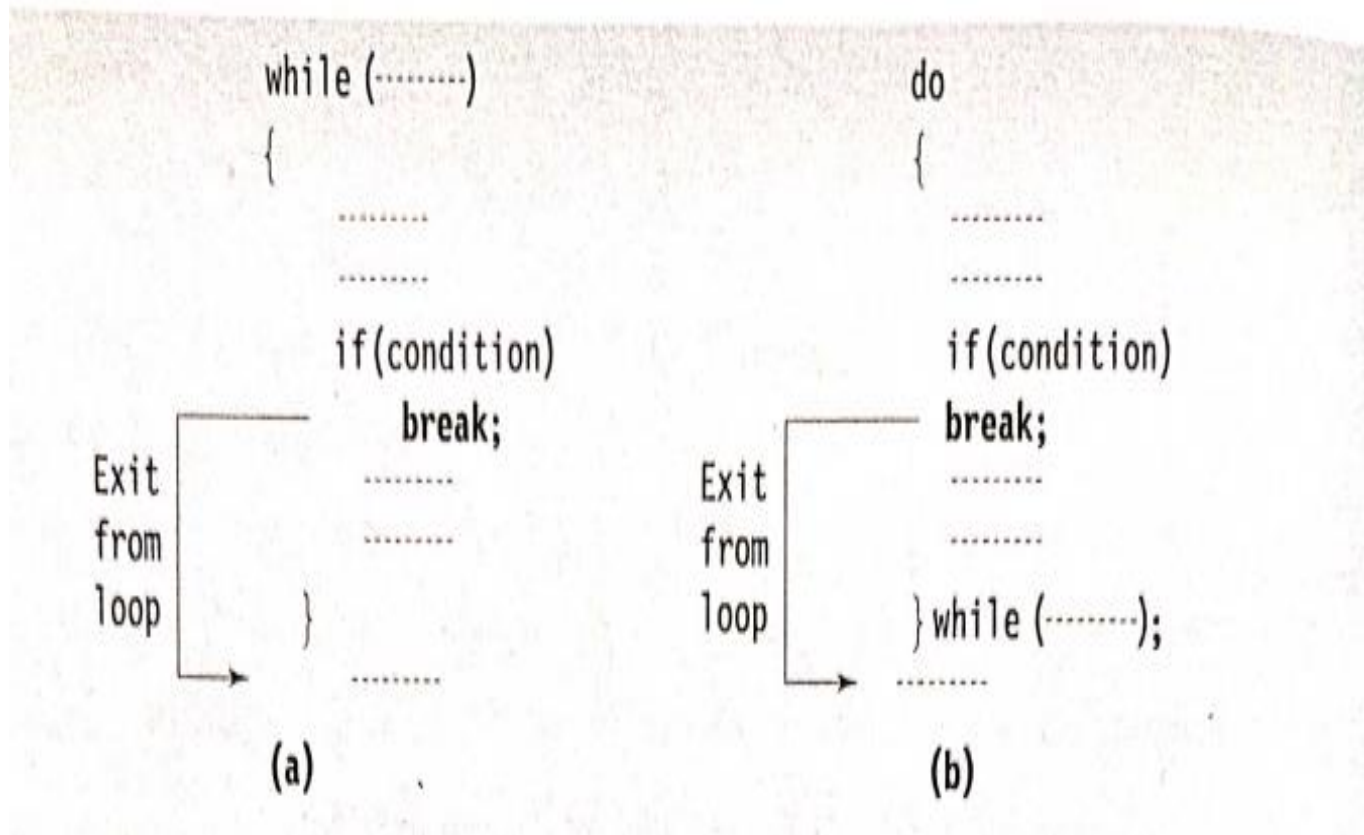
# Jumps In Loops

- Sometimes, when executing a loop it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs.
- C permits a jump from one statement to another within a loop as well as a jump out of a loop.

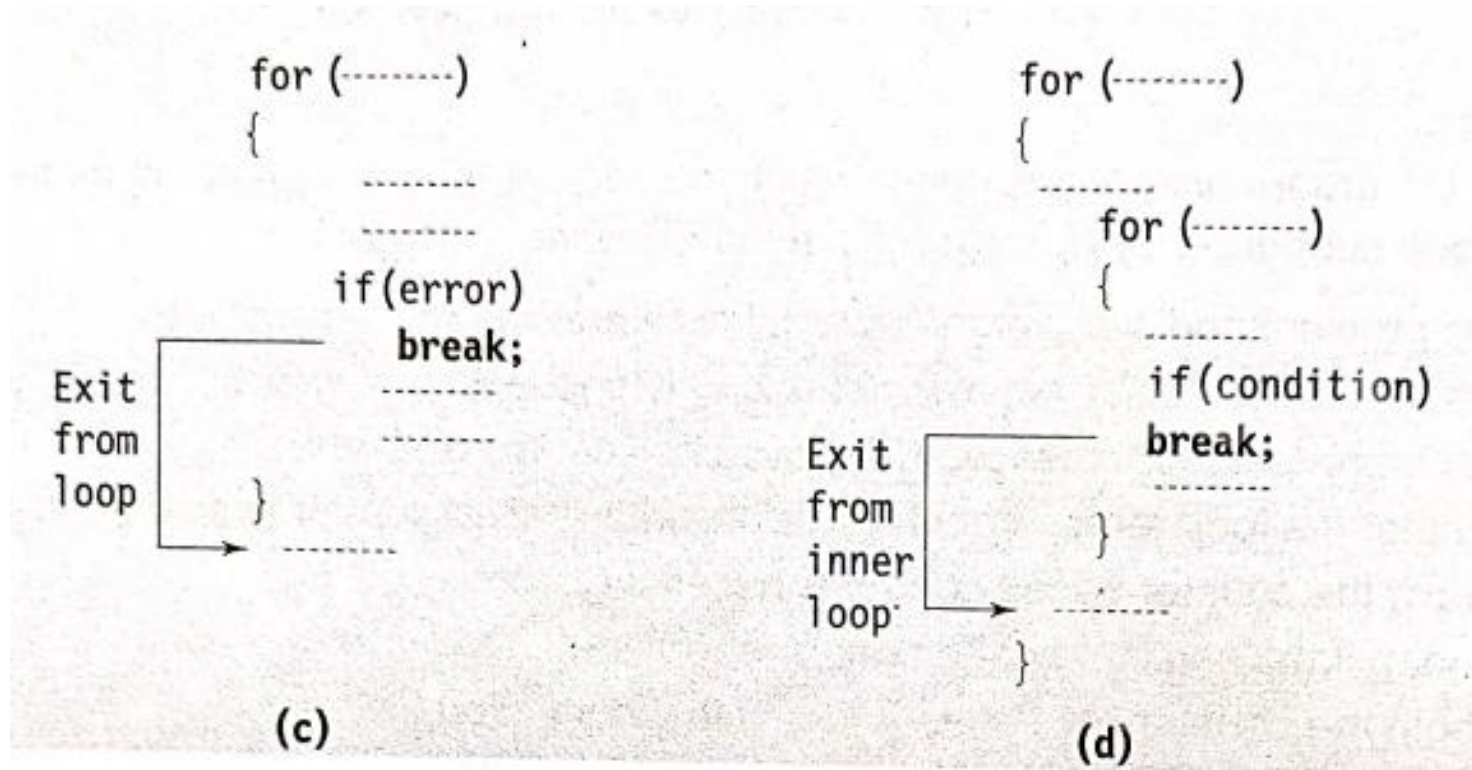
## Jumping Out of a Loop

- An early exit from a loop can be done by using the break statement or the goto statement.
- When a break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop.

# Cont...



# Cont...



- When the loops are nested, the `break` would only exit from the loop containing it. That is, the `break` will exit only a single loop.

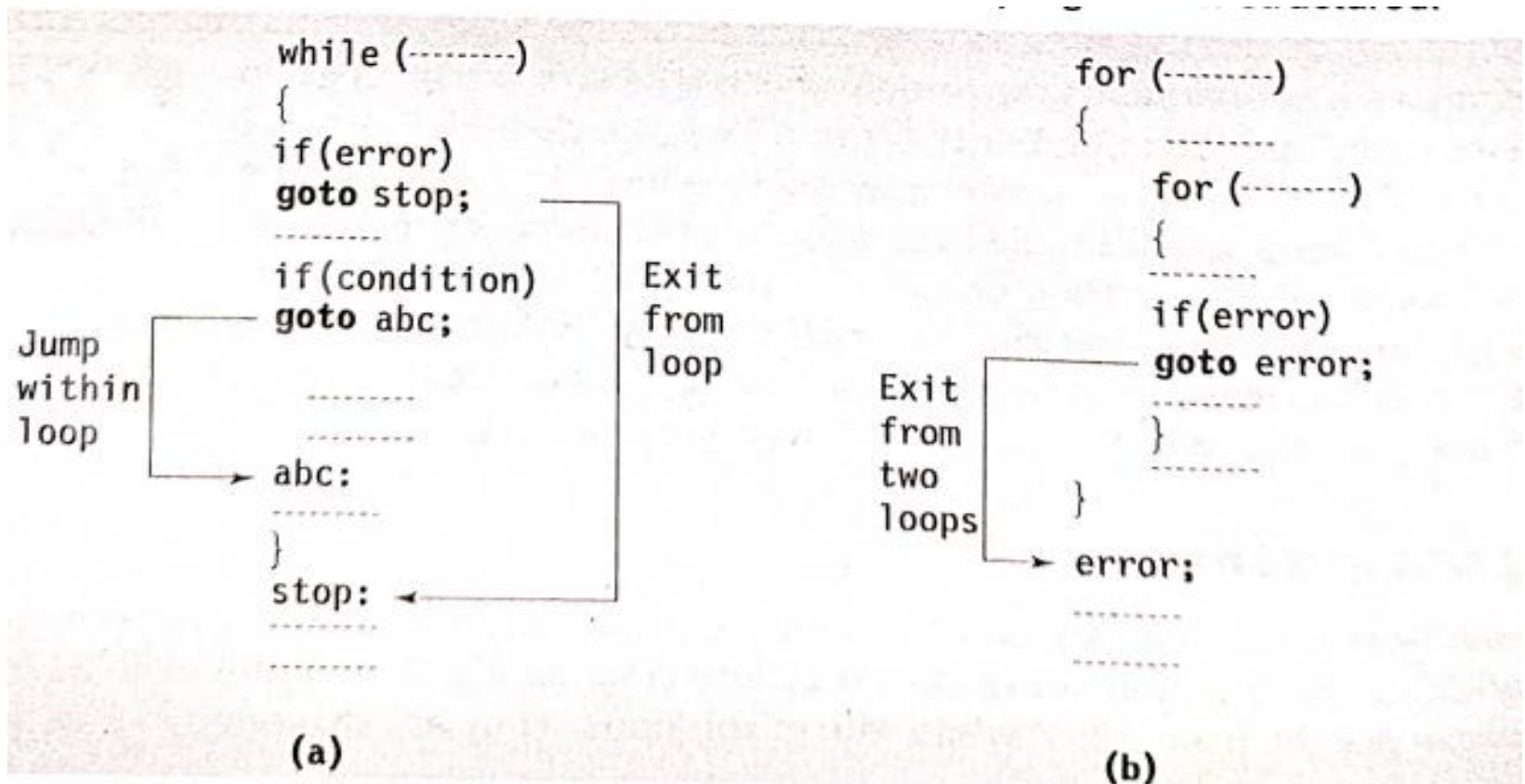
# Use of the break statement in a C program

```
#include<stdio.h>

void main()
{
 int m;
 float x, sum;
 sum = 0;
 for (m=1;m<=100 ;m++)
 {
 scanf("%f",&x);
 if (x<0)
 {
 break;
 }
 sum=sum+x ;
 }
 printf("Sum=%f\n",sum);
}
```

# Jump using goto

- goto statement can transfer the control to any place in a program, it is useful to provide branching within a loop.
- Use to exit from deeply nested loops.



Jumping within and exiting from the loops with **goto** statement

# Skipping a Part Of Loop

- During loop operations, it may be necessary to skip a part of the body of the loop under certain conditions.
- To skip some part of the body of the loop, C supports a continue statement.
- However, unlike the break which causes the loop to be terminated, the continue, as the name implies, causes the loop to be continued with the next iteration after skipping any statements in between.
- The continue statement tells the compiler,

“SKIP THE FOLLOWING STATEMENTS AND  
CONTINUE WITH THE NEXT ITERATION”

# Cont...

➤ format of continue

**continue;**

- In while and do-while loops continue causes the control to go directly to the test-condition and then to continue iteration process.
- In the case of for loop, the increment section of the loop is executed before the test-condition is evaluated.

# Cont..

```
while (test-condition)
{

 if (-----)
 continue;

}
```

(a)

```
do
{

 if (-----)
 continue;

} while (test-condition);
```

(b)

```
for (initialization; test condition; increment)
{

 if (-----)
 continue;

}
```

(c)

# Use of the continue statement in a C program

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
 int m;
```

```
 float x, sum;
```

```
 sum = 0;
```

```
 for (m=1;m<=100 ;m++)
```

```
 {
```

```
 scanf("%f",&x);
```

```
 if (x<0)
```

```
 {
```

```
 continue;
```

```
 }
```

```
 sum=sum+x ;
```

```
 }
```

```
 printf("Sum=%f\n",sum); }
```

Viranchi N Patel,DDU

# Jumping out of the Program

- we can jump out of a program by using the function `exit( )`.
- In case, due to some reason, we wish to break out of a program and return to the operating system, we can use the `exit( )` function.

```
if(test-condition)
{
 exit(0);
}
```

- The `exit( )` function takes an integer value as its argument. Normally zero is used to indicate normal termination and a nonzero value to indicate termination due to some error or abnormal condition.
- The use of `exit( )` function requires the inclusion of the header file `<stdlib.h>`.

# Use of the exit() in a C program

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void main()
```

```
{
```

```
 int m;
```

```
 float x, sum;
```

```
 sum = 0;
```

```
 for (m=1;m<=100 ;m++)
```

```
 {
```

```
 scanf("%f",&x);
```

```
 if (x<0)
```

```
 {
```

```
 exit(0);
```

```
 }
```

```
 sum=sum+x ;
```

```
 }
```

```
 printf("Sum=%f\n",sum); }
```

# CONCISE TEST EXPRESSIONS

if (expression ==0)

is equivalent

if(!expression)

if (expression! = 0)

is equivalent

if (expression)

**For example,**

if (m%5==0 && n%5==0)

is same as

if (!(m%5)&&!(n%5))

# Example

```
int a,b;
```

```
scanf("%d%d",&a,&b); //a=5,b=2
```

|                                                  |                                                |
|--------------------------------------------------|------------------------------------------------|
| <pre>if((a+b)==0) {     printf("hello"); }</pre> | <pre>if(!(a+b)) {     printf("hello"); }</pre> |
| <pre>if((a+b)!=0) {     printf("bye"); }</pre>   | <pre>if(a+b) {     printf("bye"); }</pre>      |

# Example

```
void main()
{
 int a;
 scanf("%d",&a); //0 //5
```

```
 if(a==5)
 {
 printf("hello\n");
 }
```

```
 if(a=3)
 {
 printf("bye\n");
 }
```

```
if(a= -5)
{
 printf("kite\n");
}
```

```
if(a==0)
{
 printf("hiii\n");
}
}
```

# Example

```
void main()
{
 int a;
 scanf("%d",&a); //0 //5
 if(a=0)
 {
 printf("hello\n");
 }
}
```

# Print following Pattern

\*

\* \*

\* \* \*

\* \* \* \*

\* \* \* \* \*

```
#include<stdio.h>
void main()
{
 int i,j,n;
 scanf("%d",&n); //5
 for(i=1;i<=n;i++)
 {
 for(j=1;j<=i;j++)
 {
 printf("* ");
 }
 printf("\n");
 }
}
```

# Print following Pattern

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

```
#include<stdio.h>
void main()
{
 int i,j,n;
 scanf("%d",&n); //5
 for(i=1;i<=n;i++)
 {
 for(j=1;j<=i;j++)
 {
 printf("%d ",j);
 }
 printf("\n");
 }
}
```

# Print following Pattern

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

```
#include<stdio.h>
void main()
{
 int i,j,n;
 scanf("%d",&n); //5
 for(i=1;i<=n;i++)
 {
 for(j=1;j<=i;j++)
 {
 printf("%d ",i);
 }
 printf("\n");
 }
}
```

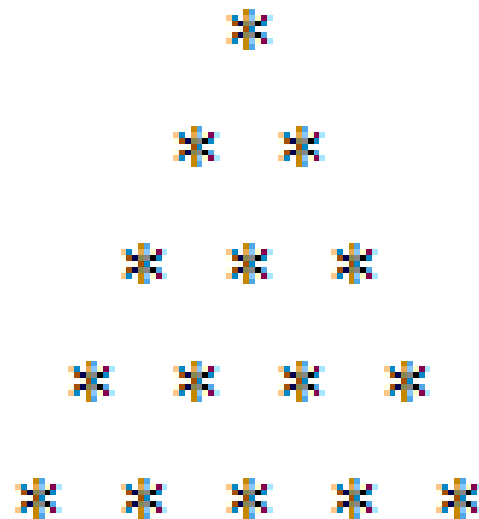
# Print following Pattern



```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf("*");
 }
 printf("\n");
 }
}
```

# Print following Pattern



```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf("* ");
 }
 printf("\n");
 }
}
```

# Print following Pattern

1  
12  
123  
1234  
12345

```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf(" %d",j);
 }
 printf("\n");
 }
}
```

# Print following Pattern

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf(" %d ",j);
 }
 printf("\n");
 }
}
```

# Print following Pattern

1  
22  
333  
4444  
55555

```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf(" %d",i);
 }
 printf("\n");
 }
}
```

# Print following Pattern

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

```
#include<stdio.h>

void main()
{
 int i,j,s,n;
 scanf("%d",&n);
 for(i=1;i<=n;i++)
 {
 for(s=n-1;s>=i;s--)
 {
 printf(" ");
 }
 for(j=1;j<=i;j++)
 {
 printf("%d ",i);
 }
 printf("\n");
 }
}
```

# Thank You